Perceptron Algorithm

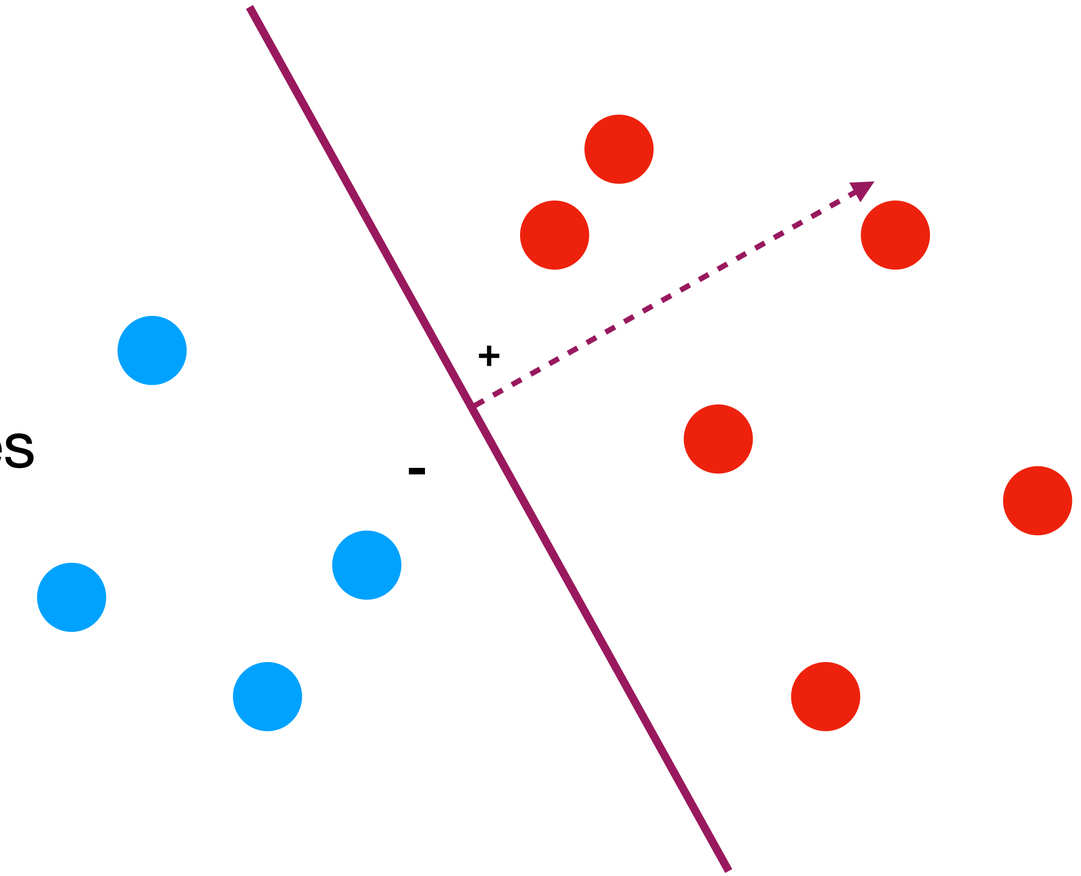
CSC 487: Deep Learning
Instructor: Jonathan Ventura

Linear classifier divides space with a hyperplane

- A linear classifier divides space into a “positive” class and “negative” class

$$z = w_1 x_1 + w_2 x_2 + b$$

$$z = \mathbf{w}^T \mathbf{x} + b$$



- **w** is a vector of “weights” that determines the line’s orientation

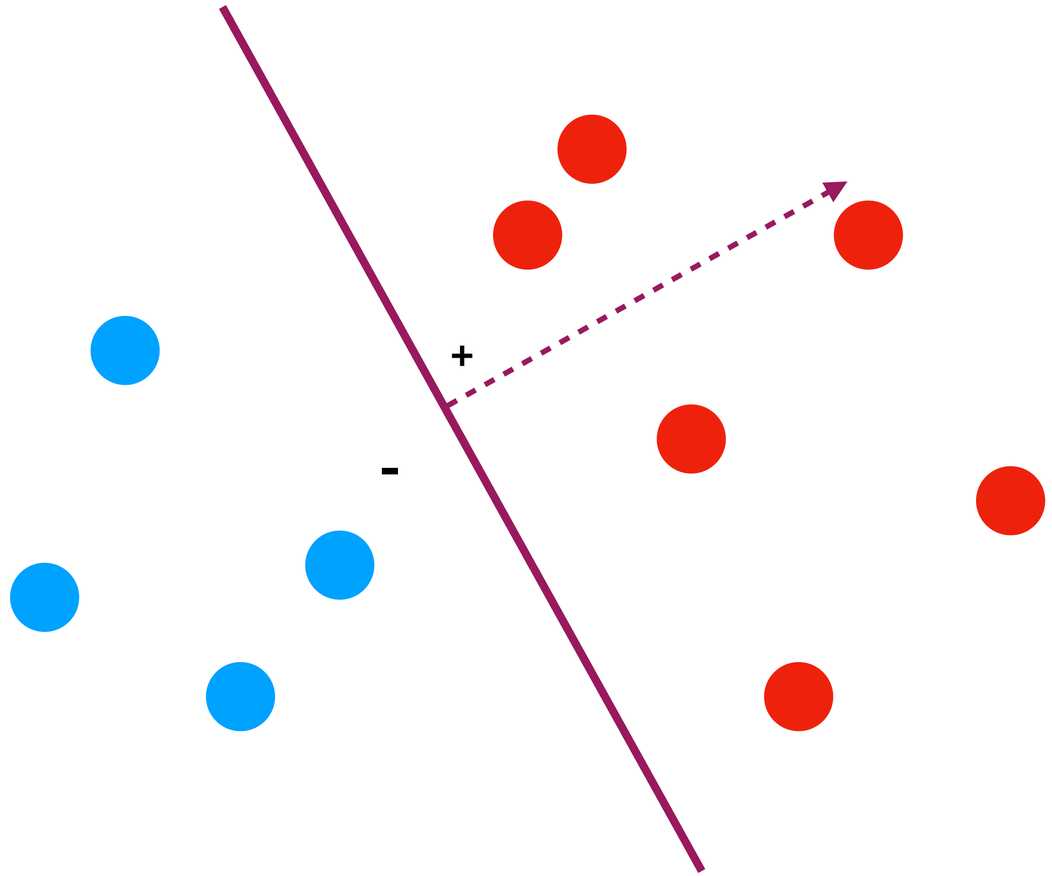
- b is a “bias” value that translates the line along the normal direction

“Hiding” the bias term

- We can “hide” the bias term by adding a 1 onto the end of the data point:

$$z = w_1 x_1 + w_2 x_2 + b1$$

$$z = \begin{bmatrix} \mathbf{w}^T & b \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}$$



Training algorithm

- **Perceptron algorithm:**

  Initialize weights to random values

  For each training point $\mathbf{x}^i$ with label $y^i \in \{-1, 1\}$:

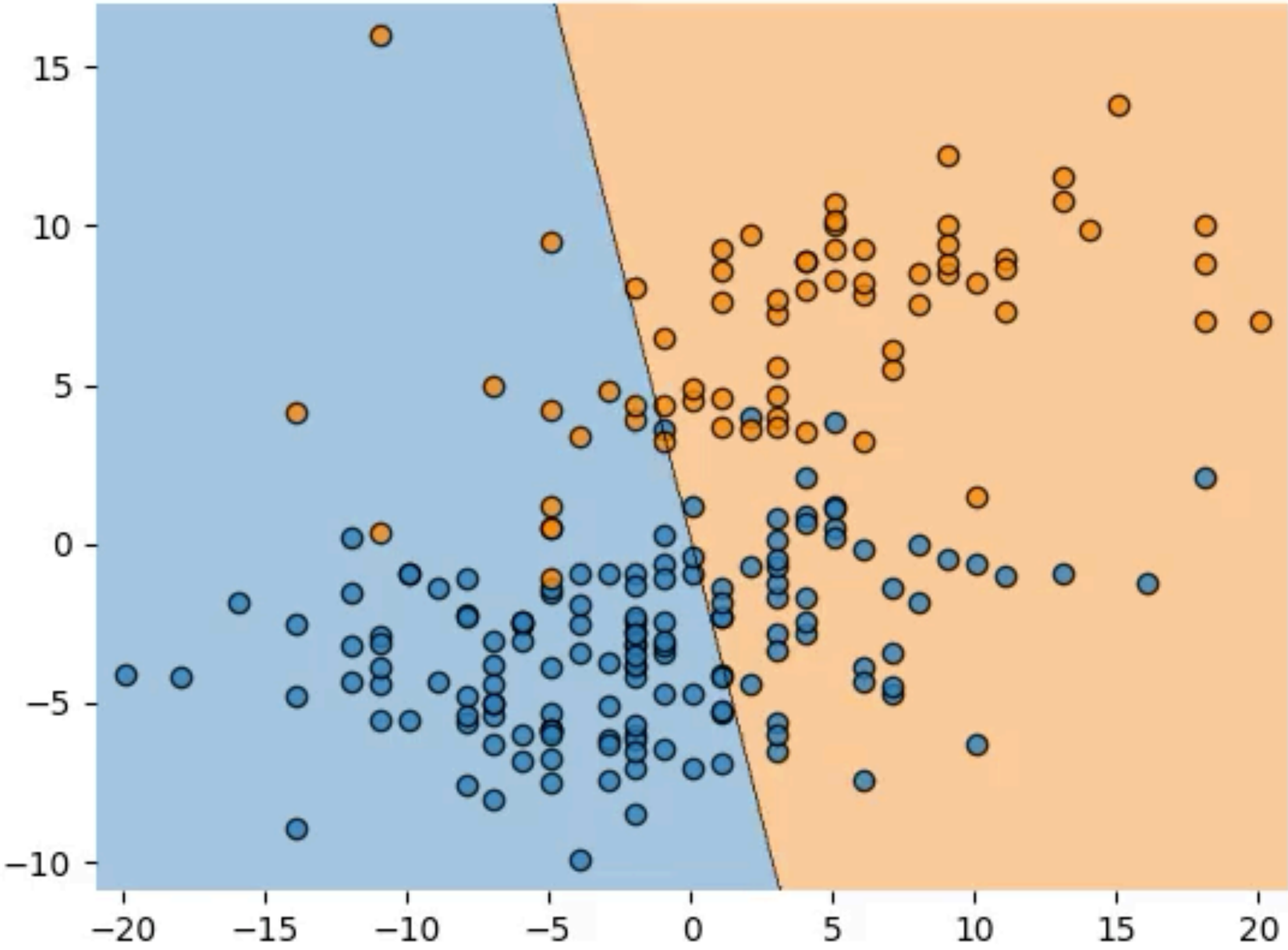
    Compute $z^i = \mathbf{w}^T \mathbf{x}^i$

    For each weight w_j:

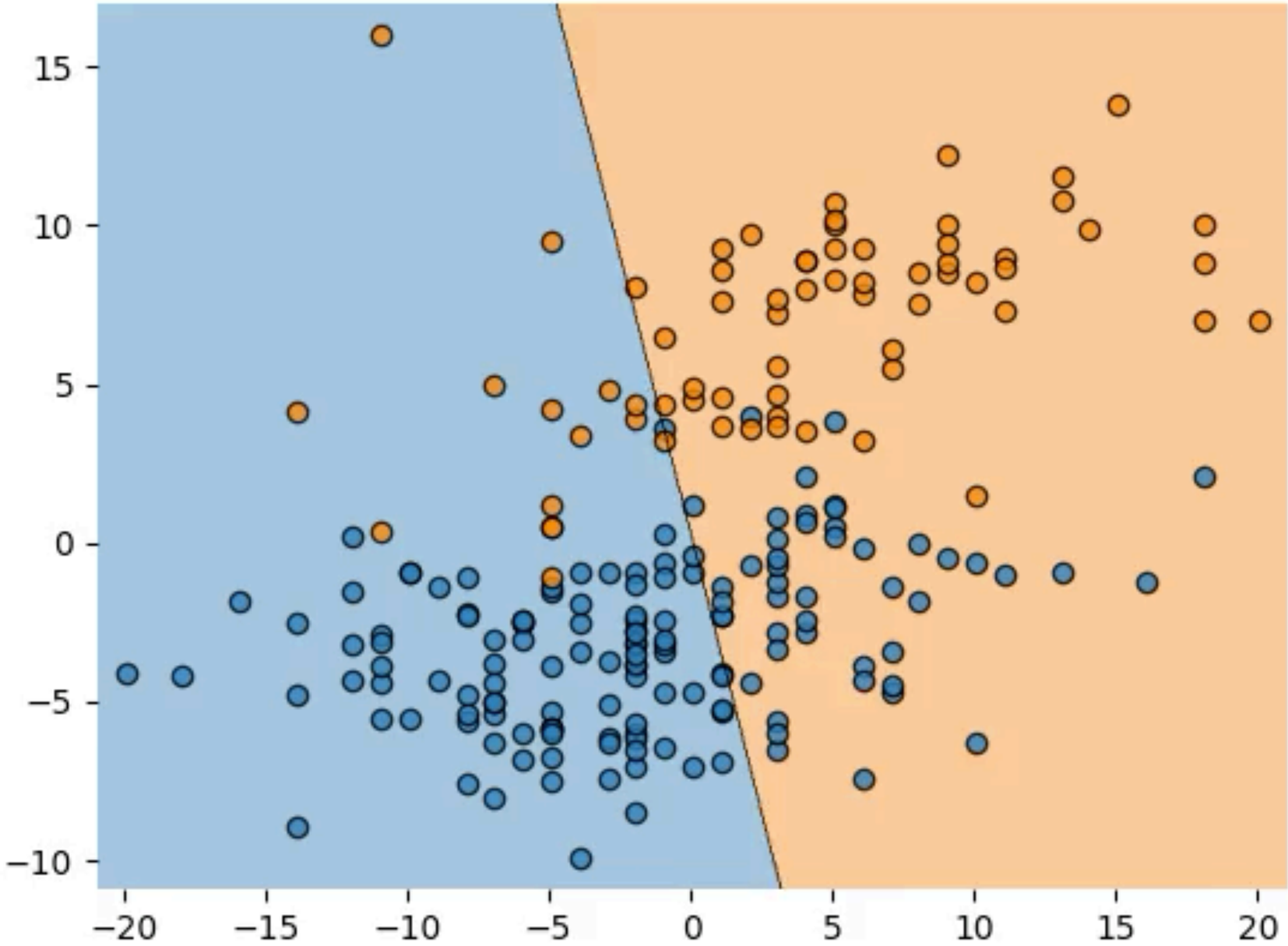
    $$w_j' = w_j + s \cdot (y^i - z^i) x_j^i$$ (s is the “learning rate”)

  Repeat until convergence

Learning rate: .001



Learning rate: .01



Vectorized update rule

For each weight w_j:

$$w_j' = w_j + s \cdot (y^i - z^i) x_j^i$$

How to write this in vectorized form?

$$\mathbf{w}' = \mathbf{w} + s \cdot (y^i - z^i) \mathbf{x}^i$$

Update rule

$$\mathbf{w}' = \mathbf{w} + s \cdot (y^i - z^i)\mathbf{x}^i$$

Why does it work?

Hint: take a look at the new value of $z^i = \mathbf{w}^T\mathbf{x}^i$ using the new weights $\mathbf{w}'$.

Update rule

$$z'^i = \mathbf{w}'^T \mathbf{x}^i$$

$$z'^i = (\mathbf{w} + s \cdot (y^i - z^i)\mathbf{x}^i)^T \mathbf{x}^i$$

$$z'^i = \mathbf{w}^T \mathbf{x}^i + s \cdot (y^i - z^i)\mathbf{x}^{iT} \mathbf{x}^i$$

$$z'^i = z^i + s \cdot (y^i - z^i)\|\mathbf{x}^i\|^2$$

If $z^i < y^i$ then $y^i - z^i > 0$ so new score will be higher.

If $z^i > y^i$ then $y^i - z^i < 0$ so new score will be lower.

Alternative update rule

$$\mathbf{w}' = \mathbf{w} + s \cdot y^i \cdot \mathbf{x}^i$$

Does this also work?

Alternative update rule

$$z'^i = \mathbf{w}'^T \mathbf{x}^i$$

$$z'^i = (\mathbf{w} + s \cdot y^i \cdot \mathbf{x}^i)^T \mathbf{x}^i$$

$$z'^i = \mathbf{w}^T \mathbf{x}^i + s \cdot y^i \cdot \mathbf{x}^{iT} \mathbf{x}^i$$

$$z'^i = z^i + s \cdot y \cdot \|\mathbf{x}^i\|^2$$

If $y^i > 0$ then new score will be higher.

If $y^i < 0$ then new score will be lower.

Where does this come from?

The first algorithm is designed to minimize a squared error “loss” function:

$$L(\mathbf{x}^i; \mathbf{w}) = \frac{1}{2}(z^i - y^i)^2$$

The second algorithm minimizes a different loss function:

$$L(\mathbf{x}^i; \mathbf{w}) = -(z^i \cdot y^i)$$

We are applying a **gradient descent** algorithm to minimize the loss.